

UI-CS-AI-2026-401628201-PRJ2

پروژه شماره ۲: جستجوی محلی (۱۴۰۵/۰۳/۴)

[مهلت انجام: ۱۴۰۵/۰۳/۱۱]

آنچه باید پاسخ دهید:

پروژه آخرین سیگنال

در سال ۲۱۴۵، پس از مجموعه‌ای از طوفان‌های خورشیدی و حملات سایبری گسترده، بسیاری از زیرساخت‌های ارتباطی زمین از بین رفته‌اند. شهرهای باقی‌مانده برای بقا به شبکه‌ای از سنسورهای هوشمند وابسته هستند که وظیفه‌ی آن‌ها شناسایی منابع انرژی، تشخیص نواحی خطرناک، ردیابی سیگنال‌های حیاتی، و حفظ ارتباط میان پایگاه‌ها است. اما مشکل بزرگی وجود دارد. تعداد سنسورها محدود است و محیط‌های عملیاتی بسیار پیچیده‌اند. برخی نواحی دارای موانع غیرقابل عبور هستند و در بعضی مناطق، اهداف مهمی وجود دارند که باید حتماً تحت پوشش قرار گیرند.

شما عضو تیم طراحی سیستم‌های خودکار هستید. وظیفه‌ی شما طراحی الگوریتمی هوشمند برای جانمایی سنسورها در محیط است تا:

- بیشترین تعداد اهداف پوشش داده شوند
- پوشش شبکه بهینه باشد
- منابع محدود به بهترین شکل استفاده شوند

برای حل این مسئله، باید از الگوریتم‌های Local Search استفاده کنید. در این پروژه، با الگوریتم‌های جستجوی محلی آشنا می‌شوید. به خصوص:

- Hill Climbing
- Simulated Annealing

هدف پروژه، حل یک مسئله‌ی بهینه‌سازی در محیط Grid World است. شما باید با استفاده از الگوریتم‌های Local Search، تعداد و مکان مناسب سنسورها را پیدا کنید تا بیشترین تعداد اهداف موجود در محیط پوشش داده شوند و منابع سیستم به صورت بهینه مصرف شوند.

این پروژه علاوه بر پیاده‌سازی الگوریتم‌ها، شامل تحلیل رفتار الگوریتم‌ها، طراحی تابع هزینه، طراحی Neighbor Function و مقایسه‌ی عملکرد روش‌های مختلف نیز می‌شود.

۱. پیش‌نیازها

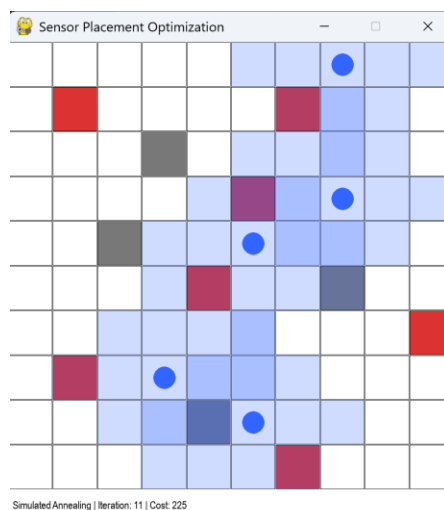
کد اولیه پروژه را از [این لینک](#) دانلود کنید.

برای اجرای محیط نیاز به نصب کتابخانه‌های زیر دارید:

- pygame
- numpy
- matplotlib

۲. محیط

محیط پروژه یک محیط شطرنجی دوبعدی با ابعاد $m \times n$ است که شامل اهداف، موانع و سنسورها می‌شود. هدف عامل، پیدا کردن بهترین مکان برای قرار دادن سنسورها است تا بیشترین تعداد هدف پوشش داده شود.



(شکل ۱: نمونه‌ای از محیط پروژه شامل ۷ هدف، ۴ مانع، ۵ سنسور و بازه‌ی تحت نظارت آن‌ها با شعاع ۲)

اجزای محیط:

- سنسورها (Sensors): نقاط دایره‌ای آبی. عامل می‌تواند تعدادی سنسور (تا سقف مشخص شده برای هر نقشه) در محیط قرار دهد. هر سنسور دارای برد مشخصی است و می‌تواند هدف‌های اطراف خود را پوشش دهد.
- اهداف (Targets): مربع‌های قرمز. هدف اصلی پروژه، پوشش دادن بیشترین تعداد Target است.
- موانع (Obstacles): مربع‌های طوسی. قرار دادن سنسور روی این خانه‌ها مجاز نیست.
- خانه‌های خالی: امکان قرار دادن سنسور روی آن‌ها وجود دارد.

۳. پیاده‌سازی

برای تسهیل پیاده‌سازی الگوریتم‌ها، بخش زیادی از محیط از قبل پیاده‌سازی شده است.

فهرست این توابع تعامل با محیط:

۱. `is_valid_position(x, y) -> bool`: بررسی می‌کند آیا مختصات موردنظر:

- داخل محیط باشد
- روی مانع نباشد

۲. `get_targets() -> list`: موقعیت تمام Targetها را برمی‌گرداند.

۳. `random_position(world) -> tuple`: یک موقعیت تصادفی معتبر در محیط تولید می‌کند.

همچنین شما در این محیط به متغیرهای `grid, sensor_range, max_sensors, row, cols` دسترسی دارید که به ترتیب تعداد ستون‌های محیط، تعداد سطرها، حداکثر تعداد سنسورهایی که در اختیار دارید، شعاع نظارت هر سنسور، و نقشه‌ی کلی محیط هستند.

آنچه شما باید پیاده کنید:

شما باید تمام منطق‌های مربوط به دو الگوریتم

- Hill Climbing
- Simulated Annealing
- بقیه الگوریتم‌ها در حالت امتیازی

را پیاده‌سازی کنید.

همچنین پیاده‌سازی توابع کمکی این الگوریتم‌ها نیز مانند توابع مربوط به گرفتن `neighbor` ها، یا محاسبه‌ی `cost` و یا هر تابع کمکی دیگری، برعهده شما است.

توجه کنید که در فایل اولیه، کلاس `local_search_base` والد بقیه‌ی کلاس‌های جستجوی محلی است و توابع مشترک را می‌توانید در این کلاس پیاده‌سازی کنید.

نکته مهم: طراحی `Neighbor Function` بخش مهمی از پروژه محسوب می‌شود و انتظار می‌رود عملیات‌هایی مانند:

- جابه‌جایی سنسور
- اضافه کردن سنسور
- حذف سنسور

در طراحی همسایگی در نظر گرفته شوند.



۴. محدودیت‌های محیط

سنسورها نباید روی مانع قرار بگیرند.
سنسورها باید داخل محیط باشند.
تعداد سنسورها می‌تواند متغیر باشد، اما نباید از مقدار `max_sensors` تعیین شده برای هر نقشه بیشتر شود.

۵. سیستم امتیازدهی محیط

پس از اجرای هر الگوریتم، گزارشی در ترمینال نمایش داده می‌شود که شامل:

- نام الگوریتم
- Cost نهایی (پایاده‌سازی تابع `cost` کاملاً برعهده‌ی شماست).
- بهترین State

است.

همچنین گزارشی تصویری از:

- روند تغییرات Cost
- تعداد Iteration ها

نمایش داده می‌شود.

نکته: دانشجویان نیازی به تغییر سیستم Visualization ندارند.

۶. خروجی الگوریتم

هر الگوریتم باید خروجی‌های زیر را برگرداند:

- `best_state`
- `best_cost`
- `evaluations`
- `states_history`

توضیح خروجی‌ها:

مقدار	توضیح
<code>best_state</code>	بهترین چیدمان سنسورها شامل تعداد و موقعیت آن‌ها
<code>best_cost</code>	بهترین <code>cost</code>
<code>evaluations</code>	روند تغییرات <code>cost</code>
<code>states_history</code>	تاریخچه‌ی <code>state</code> ها

۷. نحوه اجرا

فایل main.py را بررسی کنید. این فایل محیط را ساخته و الگوریتم‌های شما را اجرا می‌کند. الگوریتم‌های شما باید در پوشه‌ی /search پیاده‌سازی شوند. نقشه‌های مختلف در مسیر /env/maps قرار دارند.

۸. معیار کیفیت الگوریتم‌ها

توجه کنید که طراحی Neighbor Function بخش مهمی از پروژه محسوب می‌شود و انتظار می‌رود عملیات‌هایی مانند:

- جابه‌جایی سنسور
- اضافه کردن سنسور
- حذف سنسور

در طراحی همسایگی در نظر گرفته شوند.

موارد زیر در ارزیابی بررسی می‌شوند:

- کیفیت پاسخ نهایی
- توانایی خروج از Local Optimum
- سرعت همگرایی
- عملکرد روی نقشه‌های مختلف
- طراحی مناسب تابع ارزیابی
- مدیریت مناسب trade-off بین کیفیت پوشش و تعداد سنسورها

۹. تحلیل مورد انتظار

در گزارش پروژه باید موارد زیر تحلیل شوند:

- تفاوت الگوریتم‌هایی که پیاده‌سازی کرده‌اید.
- تأثیر پارامترهای هر الگوریتم
- مشکل Local Optimum
- رفتار الگوریتم‌ها روی نقشه‌های مختلف
- کیفیت Neighbor Function
- تأثیر عملیات add/remove sensor بر کیفیت جستجو

۱۰. قابلیت اضافه (امتیازی)

پیاده سازی الگوریتم های زیر:

- Genetic Algorithm
- Beam Search
- Tabu Search

آنچه باید تحویل دهید:

- کد پروژه
 - گزارش از پروژه به صورت pdf (استفاده از گیت اجباری است. لینک پابلیک ریپازیتوری خود را در این گزارش بگذارید)
- را در سامانه کوئرا بارگزاری کنید. (یک نفر از هر گروه کافی است).

هشدار: لطفا انجام پروژه را به روز پایان ددلاین موکول نکنید!

تدرست و پیروز باشید

فریاضیری منم